EXPERIMENT 14 – UNFOLDED RNN ARCHITECTURE VISUALIZATION

1. Title

Unfolded RNN Architecture Visualization

2. Aim

To build a SimpleRNN model in Keras and visualise its unfolded architecture across five time steps. To extract input, recurrent and output weights, manually calculate hidden-state values, verify them against Keras and plot the hidden-state values over time.

3. Mathematical Formulae

1. RNN Hidden State Calculation

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

2. Initial Hidden State

$$h_0 = 0$$

3. Output Calculation

$$y_t = W_{hy}h_t + b_y$$

4. RNN Weight Matrices

- W_{xh}: Input-to-hidden weight matrix.
- W_{hh}: Hidden-to-hidden recurrent weight matrix.
- W_{hy}: Hidden-to-output weight matrix.

5. Tanh Activation Function

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

6. Hidden State Across Time

For five time steps:

$$h_1 = f(W_{xh}x_1 + W_{hh}h_0 + b_h)$$

$$h_2 = f(W_{xh}x_2 + W_{hh}h_1 + b_h)$$

$$\vdots$$

$$h_5 = f(W_{xh}x_5 + W_{hh}h_4 + b_h)$$

The same weight matrices are shared across all time steps.

4. System Architecture



5. Algorithm

1. Import TensorFlow, NumPy, NetworkX and Matplotlib.

2. Define a five-word input sentence and represent each word using a numerical vector.
3. Build a SimpleRNN model with three hidden units.
4. Configure the RNN to return hidden-state outputs for all five time steps.
5. Extract the input weights, recurrent weights and bias from the SimpleRNN layer.
6. Extract the output layer weights and bias.
7. Initialise the hidden state to zero.
8. Manually calculate the hidden state at each time step using the RNN equation.
9. Compare the manually calculated hidden states with the Keras outputs.
10. Create an unfolded RNN architecture using NetworkX.
11. Draw input, hidden-state and output nodes for all five time steps.
12. Label the edges with shared weights \(W_{xh}\), \(W_{hh}\) and \(W_{hy}\).
13. Plot the hidden-state values across the five time steps.

6. Python Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import networkx as nx

# 1. Define five-word input sentence
words = ["I", "love", "deep", "learning", "models"]

# 2. Represent each word using a numerical vector
X = np.array([

  [0.2, 0.1, 0.4, 0.3],
  [0.5, 0.2, 0.1, 0.4],
  [0.3, 0.6, 0.2, 0.1],
  [0.7, 0.3, 0.5, 0.2],
  [0.4, 0.8, 0.3, 0.6]
```

```
], dtype=np.float32)

X = np.expand_dims(X, axis=0)

print("Input shape:", X.shape)

# 3. Build SimpleRNN model
inputs = tf.keras.Input(shape=(5, 4))

rnn_layer = tf.keras.layers.SimpleRNN(
    3,
    activation="tanh",
    return_sequences=True,
    name="simple_rnn"
)

hidden = rnn_layer(inputs)

outputs = tf.keras.layers.Dense(
    1,
    name="output_layer"
)(hidden)

model = tf.keras.Model(
    inputs=inputs,
    outputs=outputs
)

# 4. Compile model
model.compile(
```

```
    optimizer="adam",
    loss="mse"
)

# 5. Display model summary
model.summary()

# 6. Extract RNN weights
kernel, recurrent_kernel, bias = (
    rnn_layer.get_weights()
)

print("\nInput-to-Hidden Weights (Wxh):")
print(kernel)

print("\nHidden-to-Hidden Weights (Whh):")
print(recurrent_kernel)

print("\nHidden Bias:")
print(bias)

# 7. Extract output layer weights
output_weights, output_bias = (
    model.get_layer("output_layer").get_weights()
)

print("\nHidden-to-Output Weights (Why):")
print(output_weights)

print("\nOutput Bias:")
print(output_bias)
```

```
# 8. Get Keras hidden states
hidden_model = tf.keras.Model(
  inputs=model.input,
  outputs=hidden
)

keras_hidden = hidden_model.predict(X)

print("\nKeras Hidden States:")
print(keras_hidden[0])

# 9. Manually calculate hidden states
hidden_state = np.zeros(3)

manual_hidden_states = []

for t in range(5):

  current_input = X[0, t]

  hidden_state = np.tanh(
    np.dot(current_input, kernel)
    + np.dot(hidden_state, recurrent_kernel)
    + bias
  )

  manual_hidden_states.append(
    hidden_state.copy()
  )
```

```
manual_hidden_states = np.array(
    manual_hidden_states
)

print("\nManually Calculated Hidden States:")
print(manual_hidden_states)

# 10. Compare Keras and manual hidden states
print("\nDifference between Keras and Manual States:")

difference = np.max(
    np.abs(
        keras_hidden[0] - manual_hidden_states
    )
)

print(difference)

if np.allclose(
    keras_hidden[0],
    manual_hidden_states,
    atol=1e-5
):
    print("Both hidden states match successfully.")

else:
    print("Hidden states do not match.")

# 11. Create unfolded RNN graph
G = nx.DiGraph()
```

```
for t in range(5):

    G.add_node(
        f"x{t+1}",
        layer="input"
    )

    G.add_node(
        f"h{t+1}",
        layer="hidden"
    )

    G.add_node(
        f"y{t+1}",
        layer="output"
    )

    # Input to hidden connection
    G.add_edge(
        f"x{t+1}",
        f"h{t+1}",
        label="Wxh"
    )

    # Hidden to output connection
    G.add_edge(
        f"h{t+1}",
        f"y{t+1}",
        label="Why"
    )
```

```
    # Recurrent connection
    if t > 0:

        G.add_edge(
            f"h{t}",
            f"h{t+1}",
            label="Whh"
        )

# 12. Define node positions
pos = {}

for t in range(5):

    pos[f"x{t+1}"] = (t, 2)

    pos[f"h{t+1}"] = (t, 1)

    pos[f"y{t+1}"] = (t, 0)

# 13. Draw unfolded RNN
plt.figure(figsize=(14, 6))

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=2000,
    node_color="lightblue",
    font_size=10,
    arrows=True
```

```
)

edge_labels = nx.get_edge_attributes(
    G,
    "label"
)

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels,
    font_size=9
)

plt.title("Unfolded RNN Architecture")
plt.axis("off")
plt.show()

# 14. Plot hidden-state values
plt.figure(figsize=(10, 5))

for i in range(3):

    plt.plot(
        range(1, 6),
        manual_hidden_states[:, i],
        marker="o",
        label=f"Hidden Unit {i+1}"
    )

plt.title("Hidden State Values Across Time Steps")
```

```
plt.xlabel("Time Steps")
plt.ylabel("Hidden State Value")

plt.xticks(range(1, 6))

plt.legend()
plt.grid(True)

plt.show()
```

7. Output

```
Input shape: (1, 5, 4)
Model: "functional_1"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 5, 4)	0
simple_rnn (SimpleRNN)	(None, 5, 3)	24
output_layer (Dense)	(None, 5, 1)	4

```
 Total params: 28 (112.00 B)
 Trainable params: 28 (112.00 B)
 Non-trainable params: 0 (0.00 B)

Input-to-Hidden Weights (Wxh):
[[ 0.3348328  -0.71349114 -0.73061025]
 [-0.31012505 -0.22113818 -0.29068047]
 [ 0.85680735 -0.16694129  0.21907878]
 [-0.1780492   0.55384815 -0.5161631 ]]

Hidden-to-Hidden Weights (Whh):
[[ 0.554093    0.44903955 -0.70095974]
 [-0.31313136 -0.66777    -0.6753014 ]
 [ 0.7713169  -0.5936722   0.22939831]]

Hidden Bias:
[0. 0. 0.]

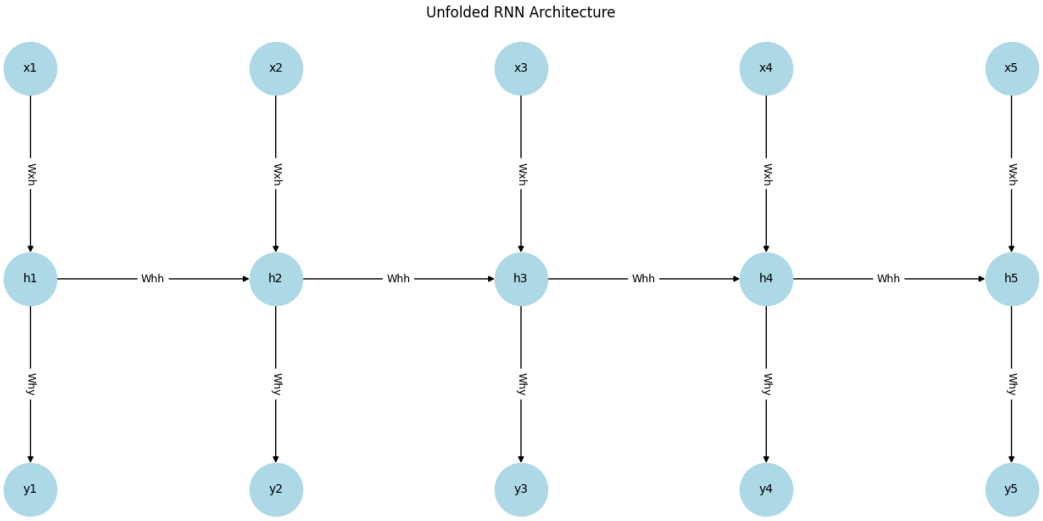
Hidden-to-Output Weights (Why):
[[0.69166875]
 [0.048962  ]
 [0.22099555]]

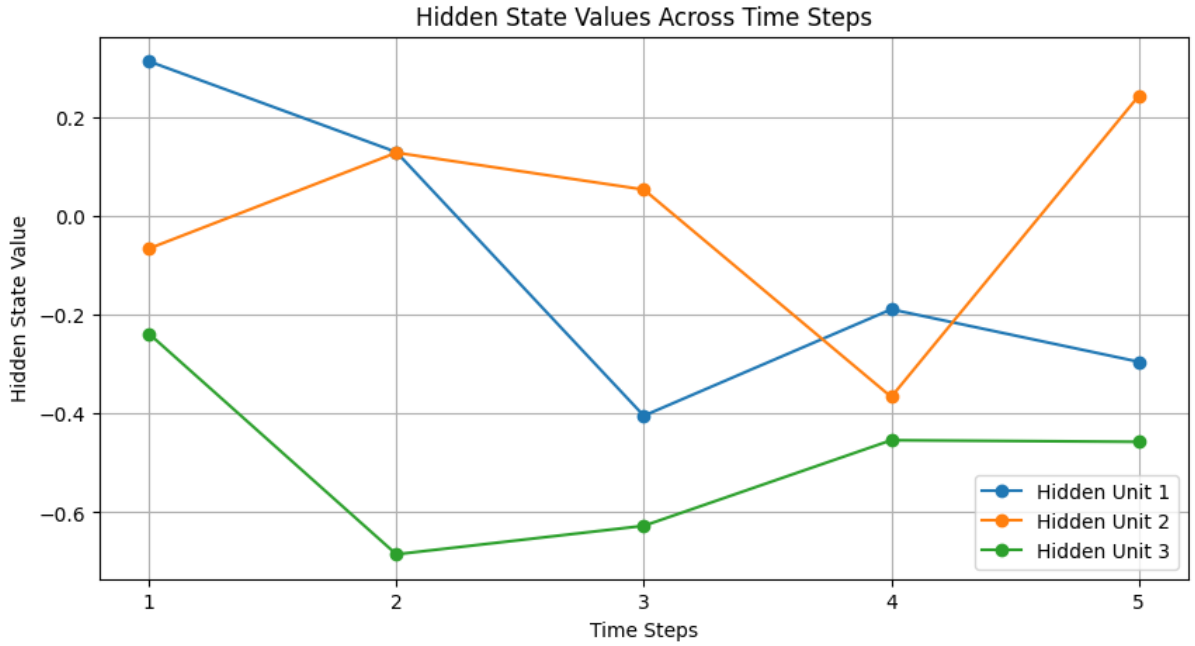
Output Bias:
[0.]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 277ms/step

Keras Hidden States:
[[ 0.3142573  -0.06534088 -0.23776849]
 [ 0.13030061  0.12905145 -0.68511945]
 [-0.40425462  0.05428266 -0.62735426]
 [-0.18846186 -0.36601868 -0.45384294]
 [-0.29480857  0.2441128  -0.45706618]]

Manually Calculated Hidden States:
[[ 0.31425729 -0.06534088 -0.23776847]
 [ 0.13030062  0.12905144 -0.68511947]
 [-0.40425459  0.0542827  -0.62735424]
 [-0.18846185 -0.36601873 -0.45384296]
 [-0.29480856  0.24411287 -0.45706617]]

Difference between Keras and Manual States:
6.568507576742988e-08
Both hidden states match successfully.
```





8. Result

Thus, a SimpleRNN model with three hidden units was successfully implemented for a five-word input sequence. The input, recurrent and output weights were extracted, and hidden states were manually calculated and verified against Keras. The unfolded RNN architecture and hidden-state values across time steps were visualised successfully.